1 Choosing the correct data structure

Problem 1.1. You have been contacted by a new company that plans to sell outdooring gear to cats, Catagonia. They want help building some tools for processing their data. In particular, they want you to develop the following functionality:
Given an array of products (each product contains a ProductID and a price), create a data structure that can be efficiently queried in the following ways:

1. Users can search for a particular product using the ID. Search HT (member/Retrieve)
2. Users can search for all products whose prices lie within some range. Report In Range
3. Users can get a list of products ordered from cheapest to most expensive. InOrder Walk.

You need to

(a) Write psuedo-code that creates the data structure.

(b) Explain what modifications you can make to make the data structure more efficient.

(c) Explain what "built-in" methods of the data structure would help with the queries.

a) Easiest DS is a BST where the key's are the product prices.

```
function BuildDS(A[],n)
    Let T be an empty BST
    for i=1 to n
        T.Insert(A[i].price, A[i].ID)
    endfor
    return T
```

b) The problem here is that it will be SLOW to search for products by ID. To fix this, lets use two different Data Structures — A BST w/ key = price, data = ID
A HT w/ key = ID, data = pointer into BST.

Problem 1.2. Write psuedocode that determines if a partially filled 9×9 Sudoku board is valid. A valid FILLED sudoku satisfies the following rules:

(a) Each row must contain the digits 1-9 without repetition.

(b) Each column must contain the digits 1-9 without repetition.

(c) Each of the nine 3×3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Only the filled cells need to be validated according to the rules mentioned above.

Idea:

```
foreach row R_i
    Check R_i for duplicates.
    check all entries ∈ [1, 9]
endfor
foreach column C_j
    Check C_j for duplicates.
    check all entries ∈ [1, 9]
endfor
foreach box B_K
    Check B_K for duplicates.
    check all entries ∈ [1, 9]
endfor
```

```
function ContainsDuplicates(A, n)
    H = empty Hash Table
    for i = 1 to n
        if H.member(A[i])
            return TRUE
        else
            H.Insert(A[i], _)
        endif
    endfor
    return FALSE
```

Note: We analyzed this in the notes!

Problem 1.3. Write psuedocode that determines if a partially filled $n^2 \times n^2$ Sudoku board is valid. A valid FILLED sudoku satisfies the following rules:

(a) Each row must contain the digits 1-n^2 without repetition.

(b) Each column must contain the digits 1-n^2 without repetition.

(c) Each of the n^2 sub-boxes of size $n \times n$ of the grid must contain the digits 1-n^2 without repetition.

Only the filled cells need to be validated according to the rules mentioned above.

The idea is the same!

Idea:

```
foreach row R_i
    Check R_i for duplicates.
    check all entries ∈ [1, n^2]
endfor
foreach column C_j
    Check C_j for duplicates.
    check all entries ∈ [1, n^2]
endfor
foreach box B_K
    Check B_K for duplicates.
    check all entries ∈ [1, n^2]
endfor
```

BC: $\Theta(1)$, R_1 has an immediate duplicate.

EC: Body of loops take $\Theta(n^2)$ time. Each runs n^2 times, so $\Theta(n^4)$.

WC: Body of loops take $\Theta(n^4)$ time; each runs n^2 times. So, $\Theta(n^6)$.

```
function ContainsDuplicates(A, n)
    H = empty Hash Table
    for i = 1 to n
        if H.member(A[i])
            return TRUE
        else
            H.Insert(A[i], _)
        endif
    endfor
    return FALSE
```

BC: $\Theta(1)$

EC: $\Theta(n)$

WC: $\Theta(n^2)$.

2 Kernel Methods for Data Structures

Problem 2.1. Suppose that we modify our implementation of a hash table using chained hashing so that the data structure also contains a stack which stores all of the keys present in the hash table. Using this knowledge, write kernel methods for the Hash Table that

(a) Returns an array containing all of the keys from the hash table (the "Key Set").

(b) Returns an array containing all of the data from the hash table (the "Value Set").

(c) Finds and returns the largest key present in the data structure.

Can you think of a better way to design this data structure that would make it even more efficient for accessing the max?

a)
```
function HashTable.KeySet()
    A = empty array of size numElements.
    i = 1
    while keyStack ≠ ∅
        A[i] = keyStack.Pop()
        i = i+1
    endwhile
    for i = n downto 1
        KeyStack.Push(A[i])
    end for
    return A
end function
```

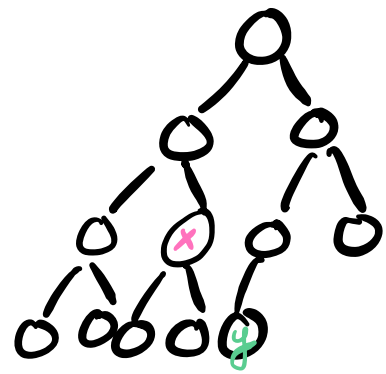
b)
```
function HashTable.ValuesSet()
    A = empty array of size numElements.
    S = empty stack
    i = 1
    while keyStack ≠ ∅
        key = keyStack.Pop()
        S.Push(key)
        j = h(key)
        Search linked list H[j] for key and retrieve data d.
        A[i] = data
        i = i+1
    endwhile
    while S ≠ ∅
        key = S.Pop()
        keyStack.Push(key)
    endwhile
    return A
endfunction
```

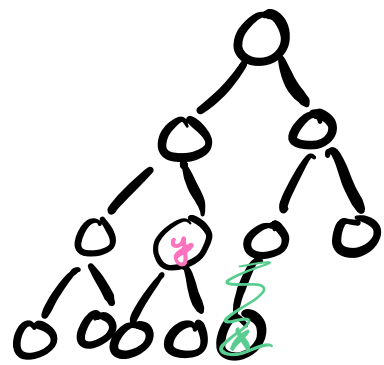
c)
```
function HashTable.maxKey()
    max = -∞
    S = empty Stack
    while keyStack ≠ ∅
        key = keyStack.Pop()
        if key > max
            max = key
        endif
        S.Push(key)
    endwhile
    while S ≠ ∅
        key = S.Pop()
        keyStack.Push(key)
    endwhile
    return key.
```

This could be improved by using a more convenient DS for the keys. A linked list would help avoid having to "rebuild" the stack. Similarly, a dynamic array would do the same. If we know the keys can be ordered, then a BST would be quite useful!

Problem 2.2. Write a kernel method called $\texttt{ExtractAt}(A[], n, i)$ for a Max-Heap of size n that removes and returns the element at location i in the array A corresponding to the Max-Heap and then fixes the heap so that the Max-Heap structured is maintained.

```
function  ExtractAt(A[], n, i)
    maxKey = A[i]
    A[i] = A[n]
    n = n - 1
    l, r, p = left(i), right(i), parent(i)
    if A[i] > A[p]
        SIFT UP
    else if (l < n and A[i] < A[l])
            or (r < n and A[i] < A[r])
        SIFT DOWN
    endif
    return maxKey
end function
```



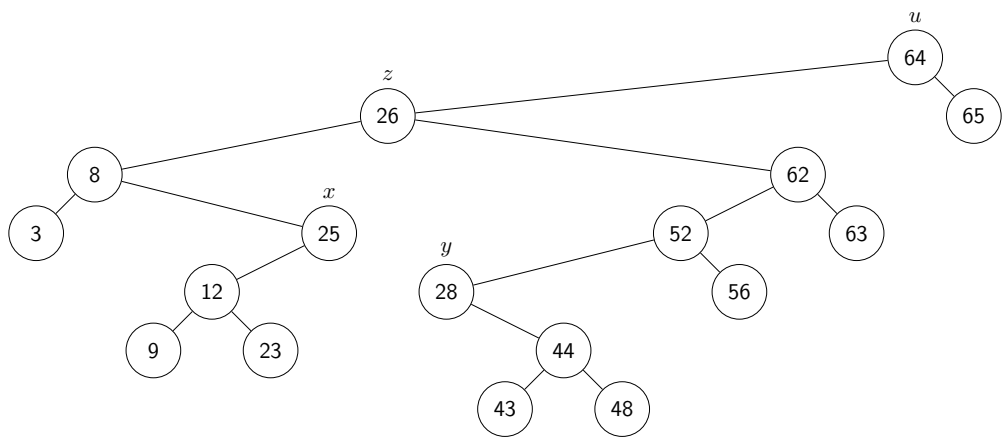


Now, decide where to sift.

SIFT UP is the procedure performed by Insert and IncreaseKey.

SIFT DOWN is the procedure performed by MaxHeapify.

Problem 2.3. Write a binary search tree method `CommonAncestor`(x, y) that finds the "least common ancester" of x and y. A node z is a common ancestor of nodes x and y if x and y are both in the subtree rooted at z. A common ancestor is a *least* common ancestor of x and y if there are no decedents of z that are also common ancestors of x and y. For example,



Note that z is the least common ancestor of x and y and u is simply a common ancestor.
Your method for finding the least common ancestor should take no more than $O(h)$ time.

Idea: Start at root and descend until "branching"

```
function BST.CommonAncestor(x,y)
  if x.key > y.key
      return commonAncestor(y,x)
  endif
  z = root of BST
  while (x.key < z.key and y.key < z.key) or (x.key > z.key and y.key > z.key)
      if x.key < z.key
          z = z.left
      else
          z = z.right
      endif
  endwhile
  return z
```

enforce $x.key \leq y.key$.

This method assumes x is not an ancestor of y and vice versa.

You can also "work upward" though this is harder.